BSCS • INNOVATION & PRODUCT IDEA ASSIGNMENT

QueueWise

Innovation & Product Idea — Smart Virtual Queue & Wait-Time Platform

NAME

Sajjid Hussain

ROLL NUMBER

5676820 (49)

GITHUB

github.com/sajjidhussainmemon4-lab/CS-352-OOP-

INNOVATION & PRODUCT IDEA ASSIGNMENT | SUBMITTED DOCUMENT

1 Student Information

Field	Detail
Name	Sajjid Hussain
Roll Number	5676820 (49)
GitHub	https://github.com/sajjidhussainmemon4-lab/CS-352-OOP-

2 Problem Statement

What problem are we trying to solve?

Walk-in service centers such as small clinics, diagnostic labs, bank branches, telecom service points, and government service counters in Pakistan largely still rely on a manual, first-come-first-served token or line system. A customer must physically arrive, take a physical token or stand in a line, and remain on the premises for an unpredictable amount of time because there is no reliable way to know how long the wait will actually be. This leads to overcrowded waiting areas, wasted commuting and waiting time, and frustration for both the customer and the staff trying to manage the crowd.

Who experiences this problem?

The problem is experienced on two sides. Customers — patients, account holders, applicants, and everyday walk-in visitors — lose productive hours sitting in waiting rooms with no visibility into their position in the queue. At the same time, the front-desk staff and owners of these small and medium service centers struggle to manage crowd flow, answer repeated "how much longer" questions, and plan staffing for peak hours, since they have no digital record of how the queue actually behaves over time.

Why is this problem important?

Time lost in physical queues is a recurring, everyday inefficiency that affects a very large number of people, particularly in urban centers where clinics, banks, and government offices are frequently crowded. For a small business, an unpredictable wait time also has a direct commercial cost: customers who feel they are wasting time are more likely to walk away before being served, and negative word-of-mouth about long queues can hurt a service center's reputation. Solving this problem therefore has both a personal-time benefit for customers and a measurable business benefit for the service centers themselves.

3 Proposed Solution

What is the proposed product?

QueueWise is a new software product — a smart virtual queue and wait-time management platform designed as an MVP for small and medium walk-in service centers. It has two connected parts: a customer-facing mobile application that lets a visitor join a service center's queue remotely and track their position in real time, and a staff-facing web dashboard that lets the service center manage the live queue, call the next customer, and view simple analytics about how the queue behaves over time.

How does it solve the problem?

Instead of physically standing in line, a customer scans a QR code posted at the service center (or opens the branch inside the app) and joins a virtual queue from their phone. The app then shows an estimated wait time, calculated from that branch's own historical service data, and sends a notification when the customer's turn is approaching, so they can arrive just in time rather than sitting in a waiting room. On the staff side, front-desk employees see a simple live list of who is waiting and can call the next customer with one tap, while the owner can later view basic analytics such as peak hours and average service time per counter.

What makes the solution useful?

The core value is that it turns an invisible, unpredictable wait into something visible and plannable, for both sides. It does not require the service center to change how it actually serves customers — it simply digitizes the queue itself, which keeps adoption effort low for small businesses while still giving them a measurable operational improvement.

> **MVP framing:** The first version deliberately focuses on the minimum feature set needed to solve the core problem — joining a queue, estimating wait time, and managing that queue from the staff side — rather than attempting a full enterprise appointment-booking suite.

4 Target Users

Primary — Everyday Customers

Patients visiting walk-in clinics and diagnostic labs, bank and telecom branch visitors, and applicants at small government service counters who want to avoid waiting in a physical line.

Secondary — Front-Desk Staff

Receptionists and counter staff at these service centers who currently manage the queue manually with tokens or a register.

Secondary — Business Owners / Managers

Owners or branch managers of clinics, small banks/franchises, and service outlets who are the paying customer of the QueueWise platform and use its analytics to plan staffing.

Out of scope (v1)

Very large, high-security institutions (major hospitals, national bank headquarters) that need custom, heavily regulated queueing systems beyond an MVP.

5 Key Features

Only the features that are directly relevant to solving the queueing problem are included, keeping the product focused as an MVP rather than an over-built system.

Feature	Description
Phone-based login	Quick registration and login using phone number and OTP verification — no lengthy sign-up form, since customers use the app briefly and infrequently.
QR / branch check-in	Customer scans a QR code at the service center (or searches the branch in-app) to join that specific branch's live queue.
AI wait-time estimation	A prediction model estimates the customer's expected wait time using that branch's historical service duration, current queue length, and time of day.
Live queue status	Real-time view of the customer's current position in the queue, updating automatically as others are served.
Push / SMS notifications	Automatic alert when the customer is a few positions away from being served, so they can arrive just in time.
Staff queue dashboard	Web dashboard for front-desk staff to view the live queue and call the next customer with a single action.
Owner analytics view	Simple charts for the business owner showing peak hours and average service time, to support staffing decisions.
Post-service feedback	A short one-tap rating after service, giving the business owner ongoing feedback on customer experience.

Deliberately excluded from v1: in-app payments, full appointment booking calendars, and multi-branch franchise management — these are reasonable future features but are not required to solve the core wait-time problem, so they are left out of this MVP.

6 Business Value

QueueWise is positioned as a business-focused idea: the paying customer is the service center itself, and the value it delivers is operational and commercial rather than purely a wellness benefit, although reducing customer waiting frustration is a welcome side effect.

How can it generate revenue?

QueueWise would use a simple software-as-a-service (SaaS) subscription model, charging each participating branch a fixed monthly fee based on the number of active service counters it manages. This keeps pricing predictable for small businesses and scales naturally as a service center grows.

Who would pay for it?

The direct paying customers are the owners or managers of the walk-in service centers — small clinics, local bank branches, telecom outlets, and similar businesses — since they are the ones who benefit from reduced crowding and better staffing decisions. The end customers who join the queue would use the app free of charge.

How can it reduce costs?

By digitizing the queue, front-desk staff spend less time manually managing tokens or answering repeated wait-time questions, freeing them to focus on actual service tasks. Better visibility into peak hours also helps owners schedule staff more efficiently instead of over-staffing quiet periods or under-staffing busy ones.

How can it improve efficiency and customer experience?

Customers who can see an accurate wait time are less likely to leave the queue out of frustration, which directly reduces lost business for the service center. Staff also benefit from a single, simple dashboard rather than a paper register, reducing errors such as calling customers out of order.

7 Feasibility

What technology could be used?

Flutter (customer app) | React.js (staff dashboard) | Node.js + Express (backend API) | PostgreSQL (core data) | Redis (live queue state) | Socket.io (real-time updates) | Firebase Auth (OTP login) | Python + scikit-learn (wait-time model) | Firebase Cloud Messaging (push alerts) | AWS EC2 / RDS (hosting)

Why these technologies are suitable

Flutter allows a single mobile codebase to target both Android and iOS, which matters for an MVP with limited development resources. Node.js and PostgreSQL are well-documented and widely supported for building a straightforward REST API and storing structured data such as branches, counters, and queue entries. Redis and Socket.io together are a standard, lightweight combination for keeping a live queue position updated in real time without constantly re-querying the main database. The wait-time predictor does not need a deep learning model: a regression model trained on service duration, time of day, and current queue length is enough to produce a genuinely useful estimate, and it can be improved later as more data is collected.

Is it possible to develop?

Yes. Every component listed above is a mature, widely used technology with strong documentation, and the overall system architecture — a mobile app and web dashboard talking to a REST/WebSocket backend with a relational database — is a well-established pattern that a small BSCS-level development team can realistically build and deploy as an MVP within an academic or early-stage project timeline.

What resources would be required?

A small team of two to four developers covering mobile, backend, and basic data/ML work; a modest cloud hosting budget for the initial pilot; and access to one or two willing pilot service centers (for example, a local clinic or telecom outlet) to collect real queue data and test the system under actual conditions.

What challenges might be faced, and how could they be handled?

Challenge	How it could be handled
Cold start — a new branch has no historical data for wait-time prediction	Use a reasonable default average service time for the first few days, then automatically switch to the trained model once enough real data has been collected.
Keeping the queue position updated in real time across many devices	Use Redis for the live queue state and Socket.io/WebSockets to push updates instantly instead of relying on customers refreshing the app.
Staff resistance to adopting a new digital tool	Keep the staff dashboard to a single, minimal screen (queue list plus a "call next" button) so it requires almost no training.
Unreliable customer arrival after being notified	Give a small grace-period buffer before marking a no-show, and allow staff to skip and recall if needed.

8 UI / Design / Diagrams

The following wireframes and diagram illustrate how the features described above come together for both the customer and the service-center staff, using consistent naming throughout.

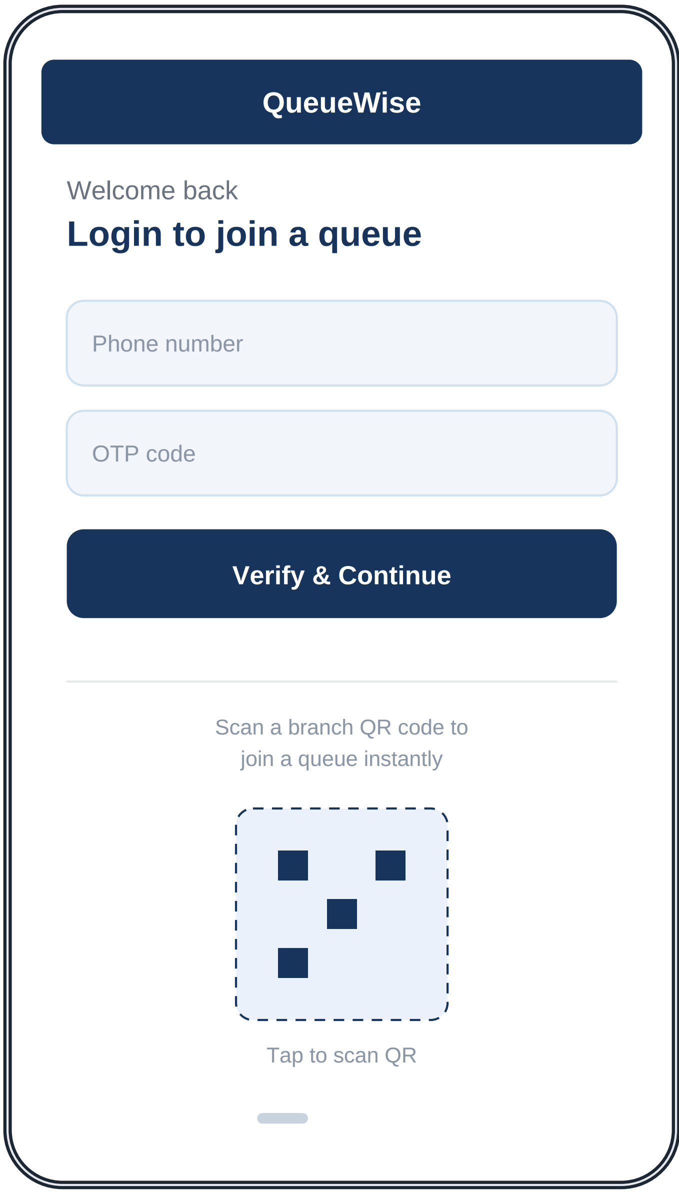

Figure 1 — Customer app: Login / Join Queue screen

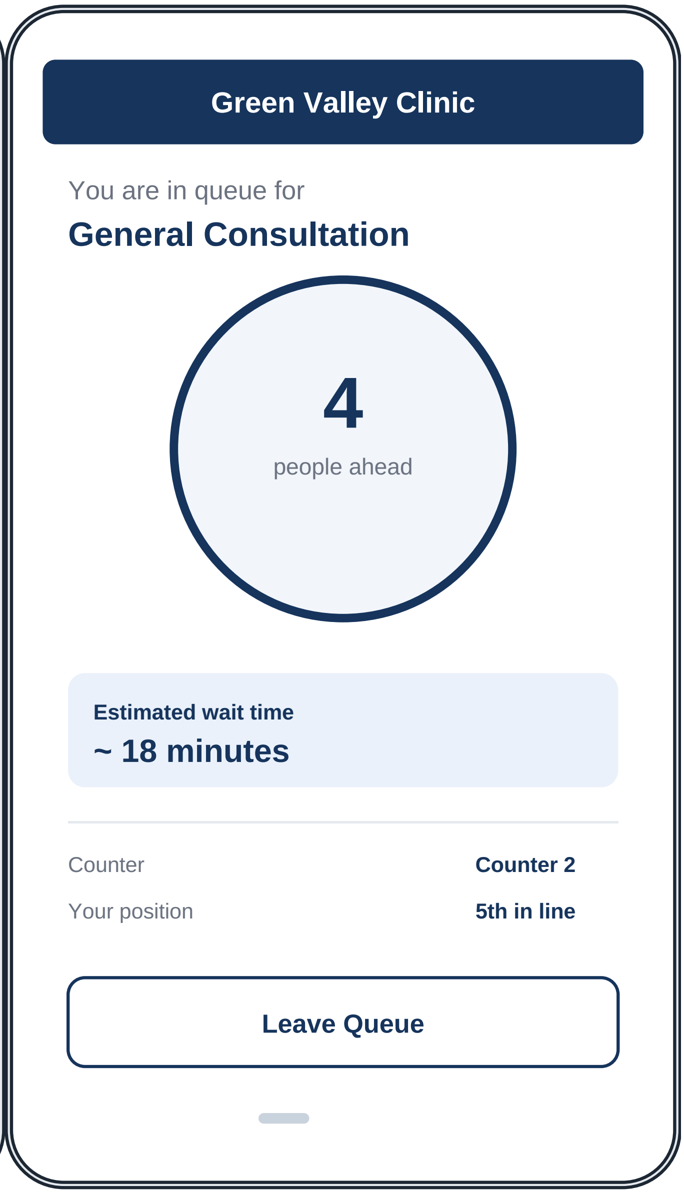

Figure 2 — Customer app: Live Queue Status screen

Figure 3 — Staff dashboard: Live Queue & daily summary

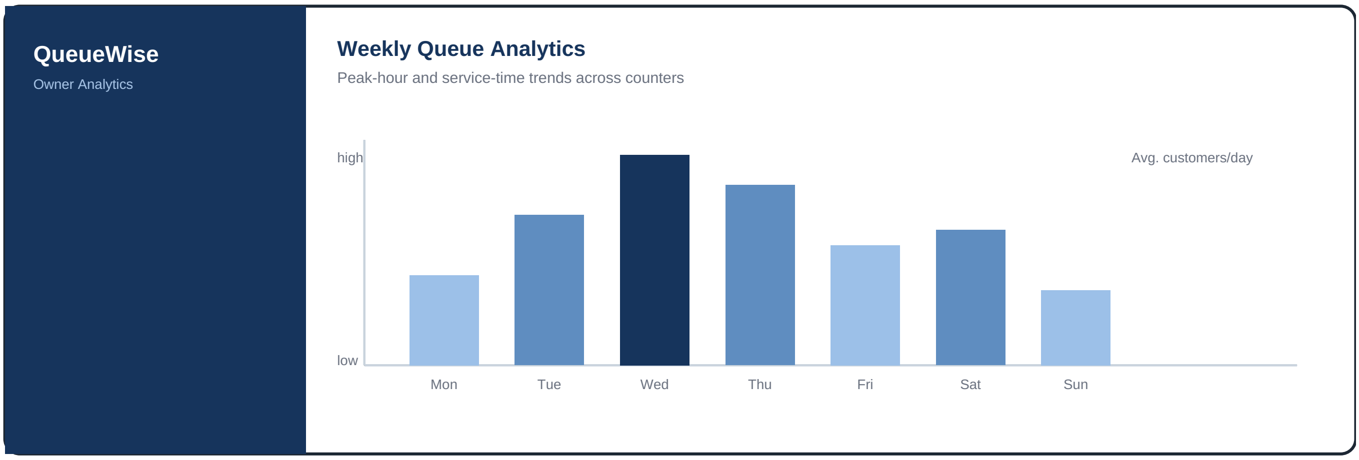

Figure 4 — Owner dashboard: Weekly analytics view

System / User-Flow Diagram

The diagram below shows how a customer, the QueueWise backend, and the staff dashboard interact from the moment a queue is joined to the moment the customer is served.

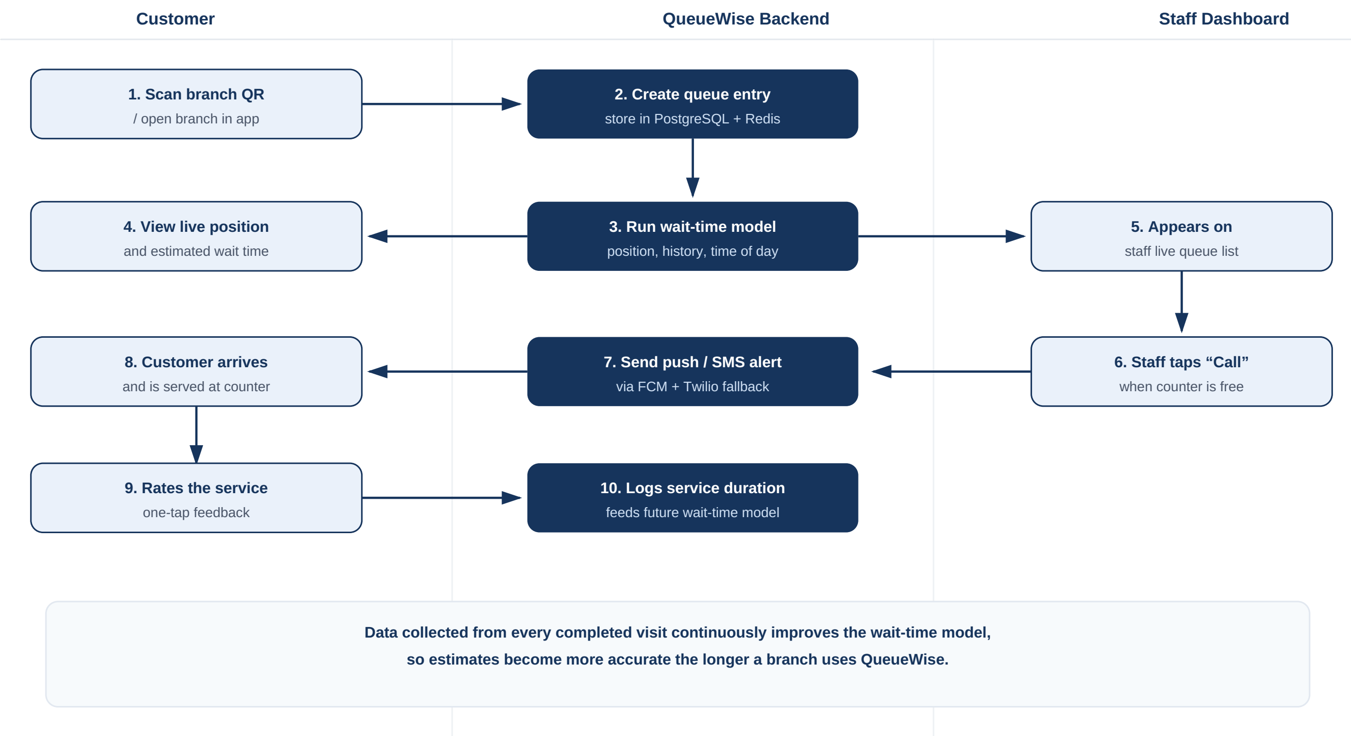

Figure 5 — End-to-end system and user-flow diagram